Symbolic Knife-Edge Diffraction for Radar Link-Budget and Detection Workflows

Derive a closed-form knife-edge diffraction model from the Fresnel-Kirchhoff integral, convert it to reusable MATLAB® functions, and feed it into radar link-budget, matched-filter detection, sensitivity, and classification workflows. The point is to answer "what does the ridge do to my radar return?" without re-integrating the diffraction physics for every altitude, range, or target hypothesis.

Workflow: Fresnel-Kirchhoff integral from first principles to symbolic tail integrals to closed-form diffraction gain and loss to `matlabFunction` code generation to ridge-obstruction geometry to two-way radar link budget to matched-filter detection to exact sensitivity to target classification features

Requires: Symbolic Math Toolbox™, Signal Processing Toolbox™

1. Fresnel-Kirchhoff Diffraction Integral

A single sharp obstruction can be modeled as a knife edge. The normalized clearance parameter ν measures how far the obstruction protrudes into the first Fresnel zone. Positive ν means the edge blocks the line of sight; negative ν means the edge is below the line of sight.

The complex field transmission coefficient for a knife edge is:

$$F(\nu) = \frac{1+j}{2}\int_{\nu}^{\infty} \exp\left(-j\frac{\pi t^2}{2}\right) dt$$

```
syms nu t real
```

Symbolic Math Toolbox evaluates the first-principles integral exactly:

```
fieldIntegral = (1 + 1i)/2 * int(exp(-1i*pi*t^2/2), t, nu, inf)
```

```
fieldIntegral =
```

$$\frac{1}{2} - \frac{\operatorname{erf}\left(\sqrt{\frac{1}{2}}\,\mathrm{i}\,\nu\,\sqrt{\pi}\right)}{2}$$

2. Closed-Form Diffraction Gain and Loss

Split the exponential into cosine and sine terms. The two tail integrals reduce directly to Fresnel cosine and sine integrals, which are efficient to evaluate numerically.

Cosine tail:

```
cTail = int(cos(pi*t^2/2), t, nu, inf)
```

```
cTail =
```

$$\frac{1}{2} - \mathrm{C}(\nu)$$

Sine tail:

```
sTail = int(sin(pi*t^2/2), t, nu, inf)
```

```
sTail =
```

$$\frac{1}{2} - \mathrm{S}(\nu)$$

Power Gain and Positive Loss

The complex multiplier contains a factor of $(1 + j)/2$, so the received power gain is:

$$|F(\nu)|^2 = \frac{1}{2}\left[\left(\frac{1}{2} - C(\nu)\right)^2 + \left(\frac{1}{2} - S(\nu)\right)^2\right]$$

Linear power gain:

```
gainLinear = simplify((cTail^2 + sTail^2)/2)
```

```
gainLinear =
```

$$\frac{\left(\mathrm{C}(\nu) - \frac{1}{2}\right)^2}{2} + \frac{\left(\mathrm{S}(\nu) - \frac{1}{2}\right)^2}{2}$$

Diffraction loss in dB, positive when the edge attenuates the field:

```
lossDb = simplify(-10*log10(gainLinear))
```

```
lossDb =
```

$$-\frac{10\log\left(\frac{\left(\mathrm{C}(\nu)-\frac{1}{2}\right)^2}{2}+\frac{\left(\mathrm{S}(\nu)-\frac{1}{2}\right)^2}{2}\right)}{\log(10)}$$

Exact sensitivity of loss to normalized clearance:

```
dLossDnu = simplify(diff(lossDb, nu))
```

```
dLossDnu =
```

$$-\frac{80\left(\cos\left(\frac{\pi\nu^2}{2}\right)\left(\mathrm{C}(\nu)-\frac{1}{2}\right)+\sin\left(\frac{\pi\nu^2}{2}\right)\left(\mathrm{S}(\nu)-\frac{1}{2}\right)\right)}{\log(10)\ ((2\,\mathrm{C}(\nu)-1)^2+(2\,\mathrm{S}(\nu)-1)^2)}$$

3. Code Generation: Deployable Diffraction Kernels

Convert the symbolic expressions into a reusable numeric kernel. Downstream tools can now call a vectorized function instead of carrying symbolic expressions or running numerical quadrature.

```
matlabFunction(gainLinear, lossDb, dLossDnu, ...
    'File', 'knifeEdgeDiffractionKernels', ...
    'Vars', {nu}, ...
    'Outputs', {'gainLinear', 'lossDb', 'dLossDnu'});
fprintf(['Generated: knifeEdgeDiffractionKernels.m\n' ...
    '  Signature: [gainLinear, lossDb, dLossDnu] = knifeEdgeDiffractionKernels(nu)\n' ...
    '  Accepts vectorized nu for altitude and geometry sweeps\n'])
```

```
Generated: knifeEdgeDiffractionKernels.m
  Signature: [gainLinear, lossDb, dLossDnu] = knifeEdgeDiffractionKernels(nu)
  Accepts vectorized nu for altitude and geometry sweeps
```

Generate Ridge Geometry Function

The Fresnel parameter comes from obstruction height relative to the line of sight and the first Fresnel-zone radius:

$$\nu = \sqrt{2}\,h/F_1, \quad F_1 = \sqrt{\frac{\lambda d_1 d_2}{d_1 + d_2}}$$

where h is positive when the ridge is above the line of sight.

```
syms targetAlt radarAlt ridgeAlt d1 d2 lambda positive
lineOfSightAlt = radarAlt + (targetAlt - radarAlt) * d1 / (d1 + d2);
obstructionHeight = simplify(ridgeAlt - lineOfSightAlt);
firstFresnelRadius = simplify(sqrt(lambda*d1*d2/(d1 + d2)));
```

Symbolic ridge-to-target geometry:

```
nuGeometry = simplify(sqrt(2) * obstructionHeight / firstFresnelRadius)
```

```
nuGeometry =
```

$$-\frac{\sqrt{2}\,\left(d_2\,\mathrm{radarAlt} - d_1\,\mathrm{ridgeAlt} - d_2\,\mathrm{ridgeAlt} + d_1\,\mathrm{targetAlt}\right)}{\sqrt{d_1}\,\sqrt{d_2}\,\sqrt{\lambda}\,\sqrt{d_1 + d_2}}$$

```
matlabFunction(nuGeometry, obstructionHeight, firstFresnelRadius, ...
    'File', 'ridgeFresnelGeometry', ...
    'Vars', {targetAlt, radarAlt, ridgeAlt, d1, d2, lambda}, ...
    'Outputs', {'nu', 'obstructionHeight', 'firstFresnelRadius'});
fprintf(['Generated: ridgeFresnelGeometry.m\n' ...
    '  Signature: [nu, h, F1] = ridgeFresnelGeometry(targetAlt, radarAlt, ridgeAlt, d1, d2, lambda)\n'])
```

```
Generated: ridgeFresnelGeometry.m
  Signature: [nu, h, F1] = ridgeFresnelGeometry(targetAlt, radarAlt, ridgeAlt, d1, d2, lambda)
```

Link-Budget Helper

The folder ships with `radarLinkBudget.m`, a conventional engineering helper that calls the generated diffraction kernel inside a two-way monostatic radar equation. Keeping this helper as readable source makes the boundary clear: symbolic expressions generate kernels, while normal workflow code consumes them.

4. Reference Scenario: S-Band Radar Behind a Ridge

Evaluate a ground radar looking toward a small aircraft whose line of sight grazes a mountain ridge. The geometry is intentionally compact: enough to be credible for radar engineers, but not so much that the symbolic workflow gets buried.

Parameter	Value	Rationale
Carrier frequency	3 GHz	S-band surveillance radar
Radar altitude	1650 m	Mountain-front ground site
Ridge altitude	1920 m	Ridge close to line of sight
Target altitude	2550 m nominal	Grazing, ridge-limited target
Horizontal range	30 km	Short regional surveillance case
Transmit power	10 kW	70 dBm pulsed transmitter
Antenna gain	36 dB	Modest dish/array gain

```
cVal = 299792458;
fcVal = 3e9;
lambdaVal = cVal / fcVal;
radarAltVal = 1650;
ridgeAltVal = 1920;
targetAltNom = 2550;
targetEastVal = 28000;
targetNorthVal = 11000;
horizontalRangeVal = hypot(targetEastVal, targetNorthVal);
d1Val = 9000;
d2Val = horizontalRangeVal - d1Val;
slantRangeVal = hypot(horizontalRangeVal, targetAltNom - radarAltVal);
azimuthDeg = mod(atan2d(targetEastVal, targetNorthVal), 360);
elevationDeg = atan2d(targetAltNom - radarAltVal, horizontalRangeVal);
transmitPowerDbm = 70;
antennaGainDb = 36;
targetRcsDbsm = 0;
noiseFloorDbm = -110;
[nuNom, hNom, f1Nom] = ridgeFresnelGeometry( ...
    targetAltNom, radarAltVal, ridgeAltVal, d1Val, d2Val, lambdaVal);
[snrNomDb, receivedPowerNomDbm, ~, lossBudgetNomDb] = radarLinkBudget( ...
    slantRangeVal, lambdaVal, transmitPowerDbm, antennaGainDb, targetRcsDbsm, noiseFloorDbm, nuNom);
```

```
oneWayLossDisplayDb = round(lossBudgetNomDb, 2);
twoWayLossDisplayDb = 2 * oneWayLossDisplayDb;
fprintf(['=== Reference Ridge-Limited Radar Geometry ===\n' ...
    'Carrier frequency:       %.1f GHz\n' ...
    'Wavelength:              %.3f m\n' ...
    'Azimuth/elevation:       %.2f deg / %.2f deg\n' ...
    'Slant range:             %.2f km\n' ...
    'Ridge clearance h:       %.2f m\n' ...
    'First Fresnel radius:    %.2f m\n' ...
    'Normalized clearance nu: %.3f\n'], ...
    fcVal/1e9, lambdaVal, azimuthDeg, elevationDeg, slantRangeVal/1e3, ...
    hNom, f1Nom, nuNom)
```

```
=== Reference Ridge-Limited Radar Geometry ===
Carrier frequency:       3.0 GHz
Wavelength:              0.100 m
Azimuth/elevation:       68.55 deg / 1.71 deg
Slant range:             30.10 km
Ridge clearance h:       0.75 m
First Fresnel radius:    25.11 m
Normalized clearance nu: 0.042
```

```
fprintf(['\n=== Nominal Link Budget ===\n' ...
    'One-way diffraction loss: %.2f dB\n' ...
    'Two-way diffraction loss: %.2f dB\n' ...
    'Received power:           %.2f dBm\n' ...
    'SNR:                      %.2f dB\n'], ...
    oneWayLossDisplayDb, twoWayLossDisplayDb, receivedPowerNomDbm, snrNomDb)
```

```
=== Nominal Link Budget ===
One-way diffraction loss: 6.39 dB
Two-way diffraction loss: 12.78 dB
Received power:           -102.89 dBm
SNR:                      7.11 dB
```

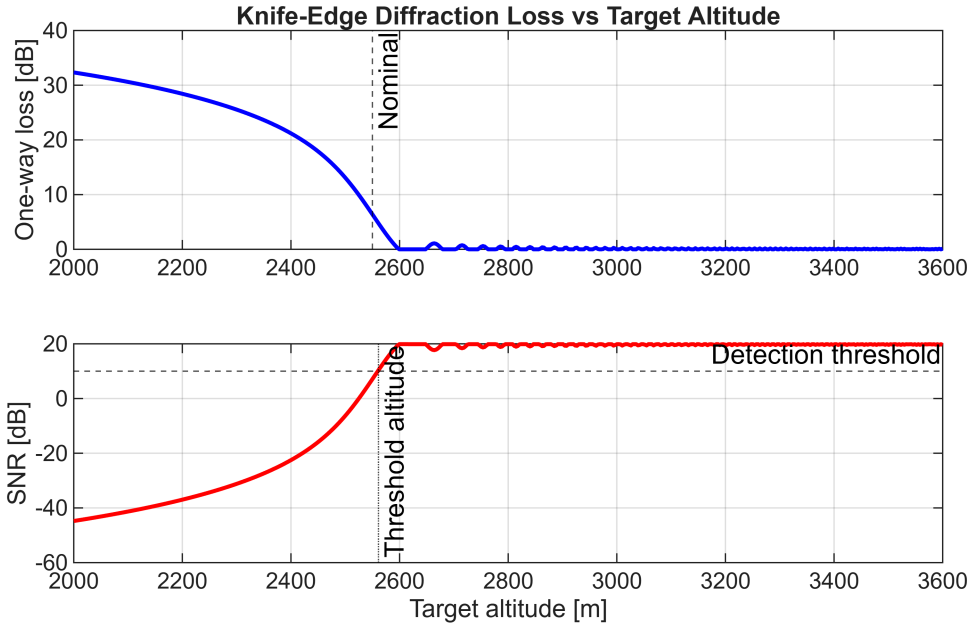
5. Altitude Sweep: Diffraction Loss to Detection Margin

The generated functions now evaluate the whole altitude design space. This is the first downstream payoff: the symbolic derivation becomes a fast link-budget sweep.

```
targetAltSweep = linspace(2000, 3600, 500);
[nuSweep, hSweep, f1Sweep] = ridgeFresnelGeometry( ...
    targetAltSweep, radarAltVal, ridgeAltVal, d1Val, d2Val, lambdaVal);
slantRangeSweep = hypot(horizontalRangeVal, targetAltSweep - radarAltVal);
[snrSweepDb, receivedPowerSweepDbm, lossSweepDb, lossBudgetSweepDb] = radarLinkBudget( ...
    slantRangeSweep, lambdaVal, transmitPowerDbm, antennaGainDb, targetRcsDbsm, noiseFloorDbm, nuSweep);
detectionThresholdDb = 10;
[~, thresholdIdx] = min(abs(snrSweepDb - detectionThresholdDb));
thresholdAlt = targetAltSweep(thresholdIdx);
fprintf(['Altitude for %.0f dB detection margin: %.0f m\n' ...
    'At that altitude: nu = %.2f, one-way loss = %.2f dB\n'], ...
    detectionThresholdDb, thresholdAlt, nuSweep(thresholdIdx), lossBudgetSweepDb(thresholdIdx))
```

```
Altitude for 10 dB detection margin: 2561 m
At that altitude: nu = -0.15, one-way loss = 4.76 dB
```

```
clf
tiledlayout(2,1)
nexttile
plot(targetAltSweep, lossBudgetSweepDb, 'b-', 'LineWidth', 1.5)
hold on
xline(targetAltNom, 'k--', 'Nominal')
hold off
ylabel('One-way loss [dB]')
title('Knife-Edge Diffraction Loss vs Target Altitude')
grid on
nexttile
plot(targetAltSweep, snrSweepDb, 'r-', 'LineWidth', 1.5)
hold on
yline(detectionThresholdDb, 'k--', 'Detection threshold')
xline(thresholdAlt, 'k:', 'Threshold altitude')
hold off
xlabel('Target altitude [m]')
ylabel('SNR [dB]')
grid on
```



6. Matched-Filter Detection Using the Derived SNR

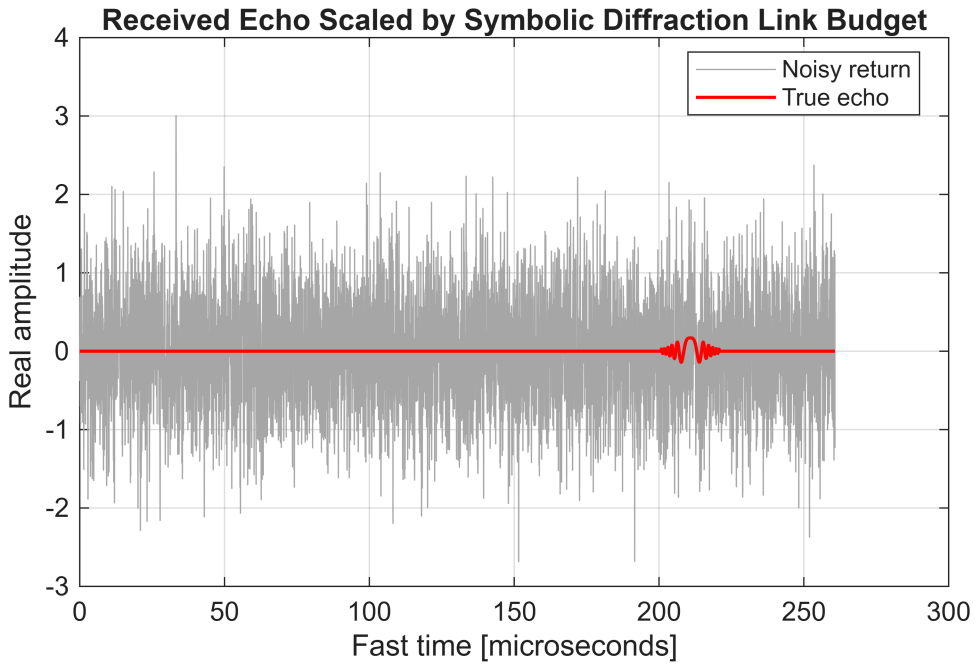
Use the nominal link budget to set the echo amplitude in a synthetic received waveform. The matched-filter output is not independent of the symbolic physics; its SNR comes from the generated diffraction and link-budget functions.

```
fsVal = 20e6;
pulseWidthVal = 20e-6;
pulseSamples = round(pulseWidthVal * fsVal);
tPulse = (0:pulseSamples-1).' / fsVal;
bandwidthVal = 2e6;
sweepRateVal = bandwidthVal / pulseWidthVal;
txPulse = exp(1i*pi*sweepRateVal*(tPulse - pulseWidthVal/2).^2);
txPulse = txPulse .* taylorwin(pulseSamples, 4, -35);
txPulse = txPulse / norm(txPulse);
roundTripDelay = 2 * slantRangeVal / cVal;
delayBins = round(roundTripDelay * fsVal);
returnSamples = delayBins + pulseSamples + 800;
```

```
rng(42)
noise = (randn(returnSamples, 1) + 1i*randn(returnSamples, 1)) / sqrt(2);
rxSignal = zeros(returnSamples, 1);
echoAmplitude = 10^(snrNomDb/20);
rxSignal(delayBins + (1:pulseSamples)) = echoAmplitude * txPulse;
rxNoisy = rxSignal + noise;
matchedFull = abs(conv(rxNoisy, conj(flipud(txPulse)), 'full'));
[matchedPeak, matchedPeakIdx] = max(matchedFull);
estimatedStartBin = matchedPeakIdx - pulseSamples + 1;
estimatedRangeVal = (estimatedStartBin - 1) / fsVal * cVal / 2;
rangeErrorVal = estimatedRangeVal - slantRangeVal;
fprintf(['=== Matched-Filter Detection ===\n' ...
    'Predicted SNR from link budget: %.2f dB\n' ...
    'Estimated range:               %.2f km\n' ...
    'Range error:                   %.1f m\n' ...
    'Matched-filter peak:           %.2f\n'], ...
    snrNomDb, estimatedRangeVal/1e3, rangeErrorVal, matchedPeak)
```

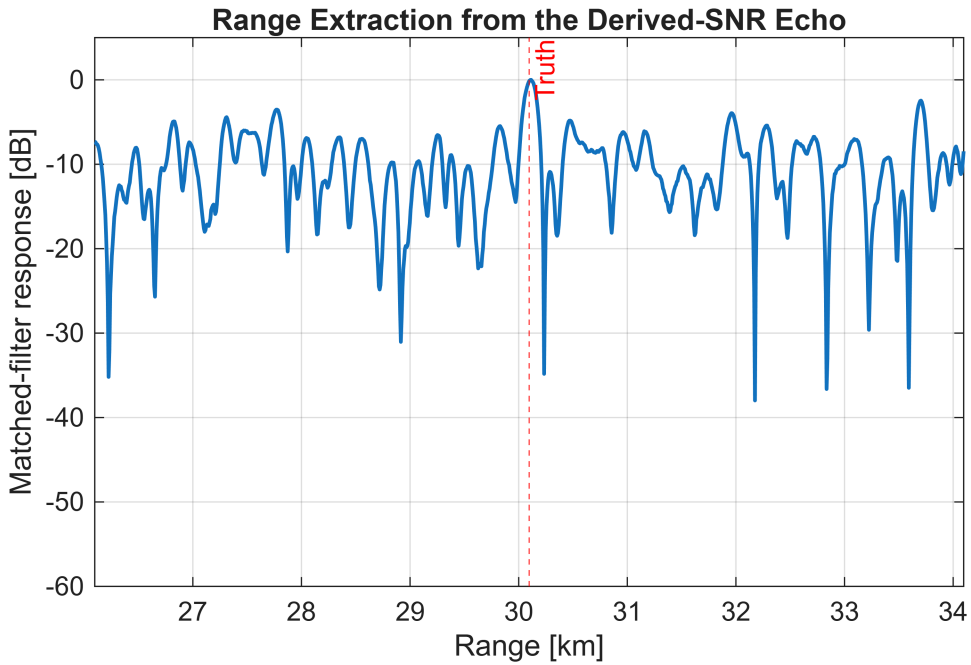
```
=== Matched-Filter Detection ===
Predicted SNR from link budget: 7.11 dB
Estimated range:               30.11 km
Range error:                   17.5 m
Matched-filter peak:           2.96
```

```
tReturnUs = (0:returnSamples-1).' / fsVal * 1e6;
clf
plot(tReturnUs, real(rxNoisy), 'Color', [0.65 0.65 0.65])
hold on
plot(tReturnUs, real(rxSignal), 'r-', 'LineWidth', 1.2)
hold off
xlabel('Fast time [microseconds]')
ylabel('Real amplitude')
title('Received Echo Scaled by Symbolic Diffraction Link Budget')
legend('Noisy return', 'True echo', 'Location', 'best')
grid on
```



Matched-Filter Range Trace

```
rangeAxisKm = ((1:numel(matchedFull)) - pulseSamples) / fsVal * cVal / 2 / 1e3;
clf
plot(rangeAxisKm, 20*log10(matchedFull / max(matchedFull) + eps), 'LineWidth', 1.4)
hold on
xline(slantRangeVal/1e3, 'r--', 'Truth')
hold off
xlabel('Range [km]')
ylabel('Matched-filter response [dB]')
title('Range Extraction from the Derived-SNR Echo')
grid on
xlim([slantRangeVal/1e3 - 4, slantRangeVal/1e3 + 4])
ylim([-60, 5])
```



7. Sensitivity Analysis: Ridge Clearance

Because the loss expression is symbolic, the derivative is exact. Convert $\partial L/\partial \nu$ to $\partial L/\partial h$ through the Fresnel geometry, then verify it with variable-precision arithmetic.

```
[~, ~, dLossDnuNom] = knifeEdgeDiffractionKernels(nuNom);
dNuDhNom = sqrt(2) / f1Nom;
dLossDhNom = dLossDnuNom * dNuDhNom;
nuValSym = vpa(sym(nuNom), 32);
dNuSym = sym(1)/1000;
lossAtNu = vpa(subs(lossDb, nu, nuValSym), 32);
lossAtNuPlus = vpa(subs(lossDb, nu, nuValSym + dNuSym), 32);
finiteDiffSensitivity = (lossAtNuPlus - lossAtNu) / dNuSym;
analyticSensitivity = vpa(subs(dLossDnu, nu, nuValSym), 32);
fprintf(['=== Diffraction Sensitivity ===\n' ...
    'dL/dnu at nominal:       %.4f dB per nu\n' ...
    'dL/dh at nominal:        %.4f dB per meter\n' ...
```

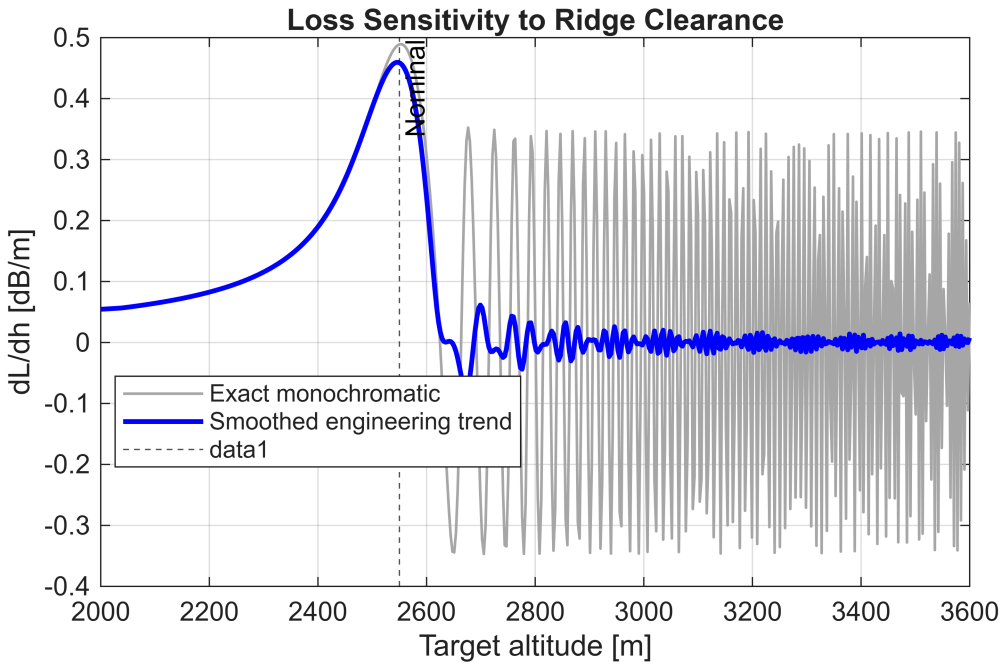
```
    'VPA analytic dL/dnu:     %s\n' ...
    'VPA finite-diff dL/dnu:  %s\n'], ...
    dLossDnuNom, dLossDhNom, char(analyticSensitivity), char(finiteDiffSensitivity))
```

```
=== Diffraction Sensitivity ===
dL/dnu at nominal:       8.6794 dB per nu
dL/dh at nominal:        0.4889 dB per meter
VPA analytic dL/dnu:     8.679413004525359883908128736865
VPA finite-diff dL/dnu:  8.679259342693350133069012822015 6
```

Sensitivity Across Altitude

The sensitivity peaks near the ridge-grazing transition, which tells the radar engineer where terrain elevation and target altitude uncertainty matter most. Above grazing, the exact single-frequency model contains real Fresnel interference fringes from constructive and destructive interference between the direct and diffracted paths. Practical radar bandwidth, ridge roughness, and atmospheric turbulence smear those rapid ripples, so the smoothed trace is the better engineering trend for margin reviews.

```
[~, ~, dLossDnuSweep] = knifeEdgeDiffractionKernels(nuSweep);
dLossDhSweep = dLossDnuSweep .* sqrt(2) ./ f1Sweep;
fringeSmoothWindow = 25;
dLossDhSmooth = movmean(dLossDhSweep, fringeSmoothWindow);
clf
plot(targetAltSweep, dLossDhSweep, 'Color', [0.65 0.65 0.65], ...
    'LineWidth', 1.0, 'DisplayName', 'Exact monochromatic')
hold on
plot(targetAltSweep, dLossDhSmooth, 'b-', 'LineWidth', 1.8, ...
    'DisplayName', 'Smoothed engineering trend')
xline(targetAltNom, 'k--', 'Nominal')
hold off
xlabel('Target altitude [m]')
ylabel('dL/dh [dB/m]')
title('Loss Sensitivity to Ridge Clearance')
legend('Location', 'best')
grid on
```



8. Classification Features from the Same Physics

Build a compact target-signature library where each feature vector is computed from target altitude, radar cross section, diffraction loss, and SNR. The classifier is deliberately simple; the teaching point is that the symbolic diffraction function now feeds a downstream analytics workflow.

```
rng(7)
classNames = ["Low Drone", "Small Aircraft", "High Aircraft", "Ridge Clutter"];
classAltMean = [2200, 2550, 3300, 2150];
classAltStd = [70, 90, 120, 45];
classRcsMean = [-12, 0, 4, -8];
classRcsStd = [2, 3, 2, 4];
nClasses = numel(classNames);
nSamplesPerClass = 45;
featureNames = ["SNR_dB", "Loss_dB", "Altitude_km", "RCS_dBsm"];
signatureFeatures = zeros(nClasses*nSamplesPerClass, numel(featureNames));
signatureLabels = strings(nClasses*nSamplesPerClass, 1);
rowIdx = 0;
```

```
for classIdx = 1:nClasses
    altSamples = classAltMean(classIdx) + classAltStd(classIdx)*randn(nSamplesPerClass, 1);
    rcsSamples = classRcsMean(classIdx) + classRcsStd(classIdx)*randn(nSamplesPerClass, 1);
    [nuSamples, ~, ~] = ridgeFresnelGeometry( ...
        altSamples, radarAltVal, ridgeAltVal, d1Val, d2Val, lambdaVal);
    rangeSamples = hypot(horizontalRangeVal, altSamples - radarAltVal);
    [snrSamples, ~, ~, lossBudgetSamples] = radarLinkBudget( ...
        rangeSamples, lambdaVal, transmitPowerDbm, antennaGainDb, ...
        rcsSamples, noiseFloorDbm, nuSamples);
    sampleRows = rowIdx + (1:nSamplesPerClass);
    signatureFeatures(sampleRows, :) = [ ...
        snrSamples(:), lossBudgetSamples(:), altSamples(:)/1e3, rcsSamples(:)];
    signatureLabels(sampleRows) = classNames(classIdx);
    rowIdx = rowIdx + nSamplesPerClass;
end
newEchoFeature = [snrNomDb, lossBudgetNomDb, targetAltNom/1e3, targetRcsDbsm];
featureMean = mean(signatureFeatures, 1);
featureScale = std(signatureFeatures, 0, 1);
featureScale(featureScale == 0) = 1;
scaledLibrary = (signatureFeatures - featureMean) ./ featureScale;
scaledEcho = (newEchoFeature - featureMean) ./ featureScale;
distances = sqrt(sum((scaledLibrary - scaledEcho).^2, 2));
[sortedDistances, sortedIdx] = sort(distances);
kNeighbors = 7;
topLabels = signatureLabels(sortedIdx(1:kNeighbors));
[sortedClasses, voteCounts] = sortClasses(topLabels, classNames);
predictedClass = sortedClasses(1);
nearestMatches = table((1:kNeighbors)', topLabels, sortedDistances(1:kNeighbors), ...
    'VariableNames', {'Rank', 'Label', 'ScaledDistance'});
disp(nearestMatches)
```

Rank	Label	ScaledDistance
1	"Small Aircraft"	0.075202
2	"Small Aircraft"	0.26586
3	"Small Aircraft"	0.32897
4	"Small Aircraft"	0.3657
5	"Small Aircraft"	0.45604

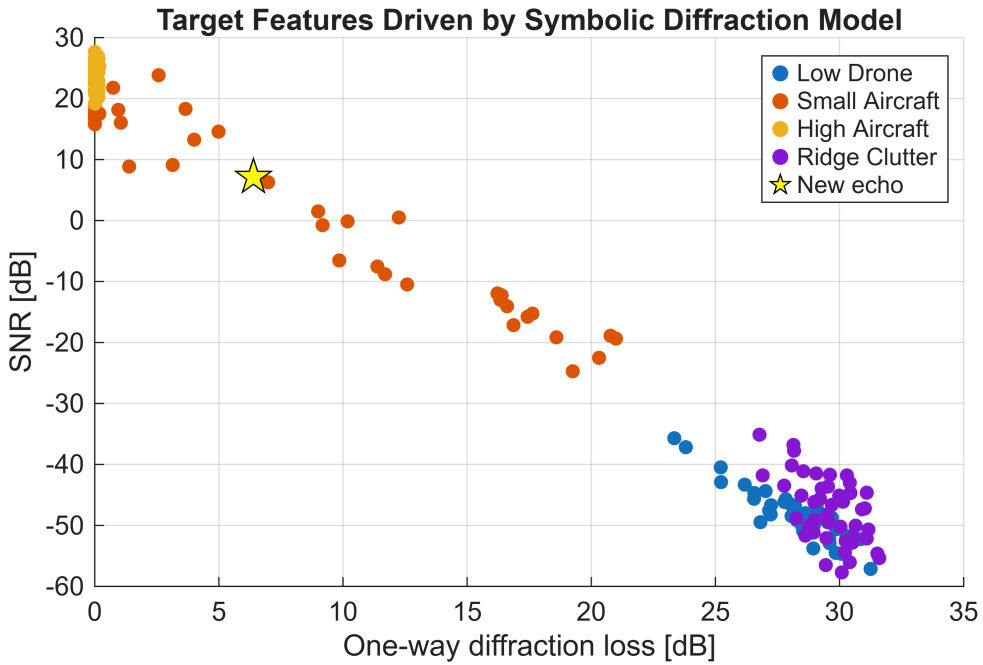
```
    6      "Small Aircraft"        0.58292
    7      "Small Aircraft"        0.59332
```

```
fprintf(['Classification result: %s\n' ...
    'Features used: %s, %s, %s, %s\n'], ...
    predictedClass, featureNames(1), featureNames(2), featureNames(3), featureNames(4))
```

```
Classification result: Small Aircraft
Features used: SNR_dB, Loss_dB, Altitude_km, RCS_dBsm
```

Visualize the Physics-Aware Feature Space

```
featureColors = lines(nClasses);
clf
hold on
for classIdx = 1:nClasses
    classMask = signatureLabels == classNames(classIdx);
    scatter(signatureFeatures(classMask, 2), signatureFeatures(classMask, 1), 28, ...
        featureColors(classIdx, :), 'filled', 'DisplayName', classNames(classIdx))
end
plot(newEchoFeature(2), newEchoFeature(1), 'kp', 'MarkerSize', 14, ...
    'MarkerFaceColor', 'y', 'DisplayName', 'New echo')
hold off
xlabel('One-way diffraction loss [dB]')
ylabel('SNR [dB]')
title('Target Features Driven by Symbolic Diffraction Model')
legend('Location', 'best')
grid on
```



9. Operational Sweep: Detection Altitude Versus Radar Power

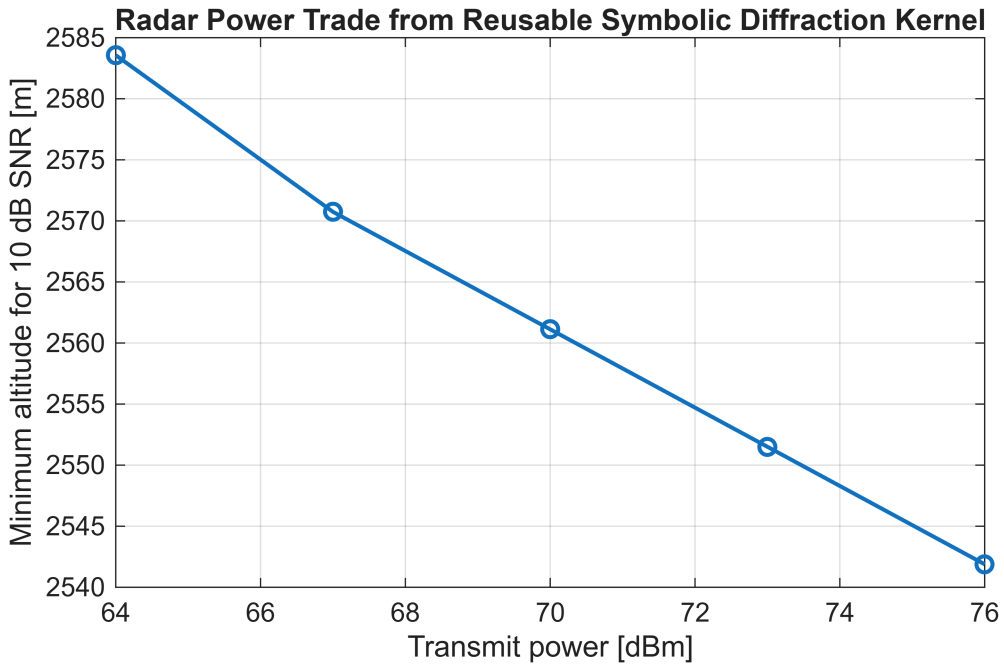
The same generated functions can be called inside an operational trade study. Here, one vectorized sweep estimates the altitude needed to maintain a 10 dB detection margin as transmitter power changes.

```
powerLevelsDbm = [64, 67, 70, 73, 76];
thresholdAltitudeByPower = zeros(size(powerLevelsDbm));
for powerIdx = 1:numel(powerLevelsDbm)
    snrPowerSweep = radarLinkBudget( ...
        slantRangeSweep, lambdaVal, powerLevelsDbm(powerIdx), antennaGainDb, ...
        targetRcsDbsm, noiseFloorDbm, nuSweep);
    firstDetectIdx = find(snrPowerSweep >= detectionThresholdDb, 1, 'first');
    if isempty(firstDetectIdx)
        thresholdAltitudeByPower(powerIdx) = NaN;
    else
        thresholdAltitudeByPower(powerIdx) = targetAltSweep(firstDetectIdx);
    end
```

```
end
tradeTable = table(powerLevelsDbm(:), thresholdAltitudeByPower(:), ...
    'VariableNames', {'TransmitPower_dBm', 'MinAltitudeFor10dB_m'});
disp(tradeTable)
```

TransmitPower_dBm	MinAltitudeFor10dB_m
64	2583.6
67	2570.7
70	2561.1
73	2551.5
76	2541.9

```
clf
plot(powerLevelsDbm, thresholdAltitudeByPower, 'o-', 'LineWidth', 1.5)
xlabel('Transmit power [dBm]')
ylabel('Minimum altitude for 10 dB SNR [m]')
title('Radar Power Trade from Reusable Symbolic Diffraction Kernel')
grid on
```



Helper Function - sortClasses

Tallies votes for each class and returns them in descending order.

```
function [sortedClasses, sortedCounts] = sortClasses(labels, classNames)
    counts = zeros(size(classNames));
    for classIdx = 1:numel(classNames)
        counts(classIdx) = sum(labels == classNames(classIdx));
    end
    [sortedCounts, order] = sort(counts, 'descend');
    sortedClasses = classNames(order);
end
```